Code-Layout, Readability and Reusability

Date	03 July 2026
Team ID	XXXXXX
Project Name	**A Comprehensive Measure of Well-Being**
Maximum Marks	5 Marks

Code-Layout, Readability and Reusability:

This document evaluates the quality of the codebase in terms of its structure, readability, and reusability. Well-organized code with proper naming conventions, comments, and modular design ensures that the project is maintainable and can be extended or reused in future development.

Code Layout Checklist:

S.No	Code Quality Parameter	Description	Followed (Yes / No / Partial)	Remarks
1	Consistent Indentation	Uniform spacing/tabs used throughout the code	Yes	Consistent 2-space (TS/TSX) and 4-space (Python) indentation maintained throughout both frontend and backend.
2	Proper File Structure	Files and folders are logically organized	Yes	Clear separation: backend/services (preprocessing, trainer, predictor) and frontend/src (components, pages, hooks, api, utils, data).
3	Meaningful Variable Names	Variables reflect their purpose clearly	Yes	Names such as life_expectancy_index, gni_per_capita, predictions_run_counter are self-descriptive.
4	Function / Method Names	Functions are descriptively named	Yes	e.g. load_clean_dataset(), make_prediction(), get_hdi_category(), train_and_save_model()
5	Code Comments	Inline and block comments explain log	Partial	Backend has useful inline comments explaining formulas; several frontend components have minimal comments and there is no root-level README describing setup/architecture.
6	Modular Design	Code is split into reusable functions/modules	Yes	Backend split into preprocessing/trainer/predictor services; frontend has a dedicated components, api, hooks, and utils layer

7	No Redundant Code	Duplicate or unused code is removed	Partial	frontend/src/utils/hdiCalc.ts (predictHDI/calculateHDI) duplicates backend HDI logic and is not called anywhere in the app; backend/inspect_dataset.py appears to be a leftover debug script overlapping preprocessing.py.
8	Error Handling	Exceptions and errors are handled gra cefully	Yes	Flask routes use try/except with proper JSON error responses and status codes; frontend uses try/catch/finally with toast notifications and graceful fallback when the API is offline.

Reusable Components / Modules:

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level (High / Medium / Low)
1	GlassCard	React / TypeScript (TSX)	Reused across Home, Predict, Dashboard, History, ModelInfo and About pages as the base card container.	High
2	StatCard (built on GlassCard + AnimatedCounter)	React / TypeScript (TSX)	Used for KPI tiles on the Dashboard page.	Medium
3	ProgressRing	React / TypeScript (TSX)	Used to render the animated HDI gauge on the Predict page.	Medium
4	AnimatedCounter	React / TypeScript (TSX)	Used inside StatCard for animated numeric displays.	Medium
5	apiClient (Axios instance, api.ts)	TypeScript	Shared base client imported by health.ts, countries.ts, analytics.ts, model.ts and prediction.ts.	High

Overall Code Quality Assessment:

Aspect	Rating (1-5)	Comments
Code Layout & Structure	4	Clean, logically separated folders on both backend (services layer) and frontend (components/pages/hooks/api/utils); consistent naming conventions across the stack.
Readability	4	Descriptive variable and function names throughout; backend formulas are well commented. Some frontend page components (e.g. Predict.tsx) are large and would benefit from further decomposition.
Reusability	4	Strong reuse of UI primitives (GlassCard, StatCard, AnimatedCounter) and a shared Axios client on the frontend, and a shared preprocessing module on the backend.
Documentation / Comments	3	Backend code has helpful inline comments, but React components have minimal documentation.
Overall Score	4	A functionally solid, well-structured full-stack project with good modularity and error handling